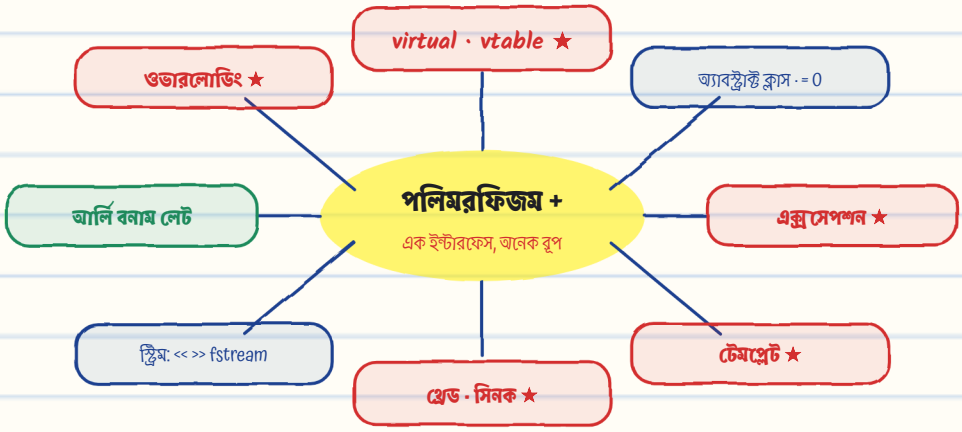
পলিমরফিজম, এক্সসেপশন, টেমপ্লেট ও থ্রেড

ভার্চুয়াল · এক্সসেপশন · টেমপ্লেট · I/O · থ্রেড: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে ধারণার ওপর বাকি সব দাঁড়িয়ে

➜ সিলেবাসের যে অংশ

OOP সিলেবাসের পরের অংশ: "*Polymorphism: overloading, abstract classes, virtual functions and overriding; Exceptions; Object Oriented I/O; Template functions and classes; Multi-threaded Programming.*" ক্লাস, অ্যাক্সেস ও *static* মেম্বার প্রথম লেসনে; কনস্ট্রাক্টর ও ইনহেরিট্যান্স দ্বিতীয় লেসনে।

হাই-ইল্ড তালিকা

① ওভারলোড: কম্পাইল টাইম; ওভাররাইড: রান টাইম
② শুধু রিটার্ন টাইপে ওভারলোড হয় না
③ ওভারলোড নয়: `::` `.` `.*` `?:` `sizeof`
④ Java-র মেথড ডিফল্টভাবে ভার্চুয়াল
⑤ ≥ ১টি *pure virtual* ⇒ অ্যাবস্ট্রাক্ট
⑥ পলিমরফিক বেসে *virtual* ডেস্ট্রাক্টর
⑦ `finally` এরর হোক বা না হোক চলে
⑧ *checked* = কম্পাইলার বাধ্য করে
⑨ টেমপ্লেট: কম্পাইল-টাইমে কপি
⑩ `start()` = নতুন থ্রেড; `run()` নয়

সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

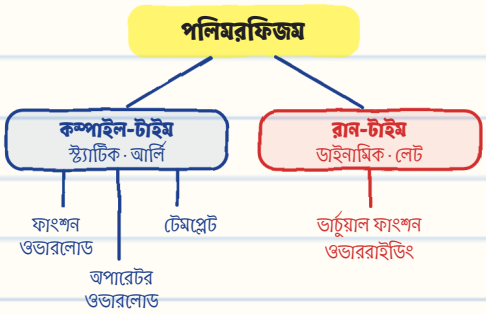
➜ সূচিপত্র

✓ পলিমরফিজম ও ওভারলোডিং ... পৃ. ২
✓ ভার্চুয়াল ফাংশন ও ওভাররাইডিং ... পৃ. ৩
✓ অ্যাবস্ট্রাক্ট ক্লাস, ভার্চুয়াল ডেস্ট্রাক্টর ... পৃ. ৪
✓ এক্সসেপশন: C++ ও Java ... পৃ. ৫
✓ টেমপ্লেট ও জেনেরিক্স ... পৃ. ৬
✓ অবজেক্ট ওরিয়েন্টেড I/O: স্ট্রিম ... পৃ. ৭
✓ থ্রেড: তৈরি ও জীবনচক্র ... পৃ. ৮
✓ সিনক্রোনাইজেশন ও ডেডলক ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ পলিমরফিজম: কম্পাইল-টাইম বনাম রান-টাইম; ওভারলোডিং

সবচেয়ে বেশি আসে ★

সংজ্ঞা পলিমরফিজম *(Polymorphism)* ("এক ইন্টারফেস, অনেক রূপ") একই নামকে জড়িত টাইপ অনুযায়ী ভিন্ন কাজ করতে দেয়। **বাইন্ডিং *(Binding)*** = একটি কলকে ফাংশন বডির সঙ্গে যুক্ত করা। আর্লি (স্ট্যাটিক) বাইন্ডিং হয় কম্পাইল টাইমে — ওভারলোডিং ও টেমপ্লেট। লেট (ডাইনামিক) বাইন্ডিং হয় রান টাইমে — বেস পয়েন্টার বা রেফারেন্সের মাধ্যমে ওভাররাইডিং।



ফাংশন ওভারলোডিংয়ের নিয়ম একই নাম, একই স্কোপ; প্যারামিটার তালিকা ভিন্ন হবে **সংখ্যা**, **টাইপ** বা **ক্রমে**। শুধু রিটার্ন টাইপ যথেষ্ট নয়: `int f(int)` + `double f(int)` = কম্পাইল এরর। কম্পাইলারের ক্রম: হুবহু মিল → প্রমোশন (`char→int`, `float→double`) → স্ট্যান্ডার্ড কনভার্শন → ইউজার কনভার্শন; দুটি সমান ভালো মিল = **অস্পষ্টতা *(ambiguity)* এরর**। *Java*-তে সাবক্লাসও উত্তরাধিকারসূত্রে পাওয়া মেথড ওভারলোড করতে পারে।

দুই পরিবার, চার হাতিয়ার

বিষয়	কম্পাইল-টাইম পলিমরফিজম	রান-টাইম পলিমরফিজম
অন্য নাম	স্ট্যাটিক, আর্লি বাইন্ডিং	ডাইনামিক, লেট বাইন্ডিং
কে ঠিক করে	আর্গুমেন্ট তালিকা / টাইপ	অবজেক্টের আসল টাইপ
কৌশল	ওভারলোডিং, টেমপ্লেট	`virtual` + ওভাররাইডিং
ইনহেরিট্যান্স লাগে?	না	হ্যাঁ

```
class Complex {
  double re, im;
public:
  Complex(double r=0, double i=0)
      : re(r), im(i) {}
  Complex operator+(const Complex& c) const {
    return Complex(re + c.re, im + c.im);
  }
};
Complex a(1,2), b(3,4);
Complex c = a + b;  // a.operator+(b) → 4+6i
```

সমাধানকৃত উদাহরণ ✍ কোন f? `void f(int); void f(double);`
`f('a')` → প্রমোশন → *f(int)*
`f(2.5f)` → *float* প্রমোট হয় → *f(double)*
`f(5L)` → *long* দুটোতেই কনভার্ট হয় → **অস্পষ্ট**

C++ অপারেটর	অপারেটরসমূহ
ওভারলোড করা যায় না	`::` · `.` · `.*` · `?:` · `sizeof` · `typeid` · `alignof` · `_cast`-গুলো · `#` `##`
কেবল **মেম্বার** হিসেবে	`=` · `()` · `[]` · `->` (এবং কনভার্শন অপারেটর)
সাধারণত ***friend***	স্ট্রিমের `<<` `>>`; প্রতিসম `a + 2` / `2 + a`

মনে রাখার কৌশল যে পাঁচটি ছোঁয়া যায় না: "***Scope, Dot, Dot-star, Question, Size***" — `::` `.` `.*` `?:` `sizeof`।

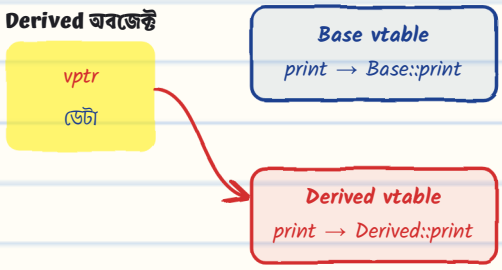
পরীক্ষার ফাঁদ *Java*-তে **ইউজার অপারেটর ওভারলোডিং নেই**; *String*-এর `+` বিল্ট-ইন। `f(int, int = 0)` + `f(int)`: `f(5)` অস্পষ্ট।

নিয়ম প্রিসিডেন্স, অ্যাসোসিয়েটিভিটি ও অপারেন্ড-সংখ্যা বদলায় না; নতুন অপারেটর (`**`) নয়; একটি অপারেন্ড ইউজার টাইপ হতেই হবে।

২ ভার্চুয়াল ফাংশন, vtable ও ওভাররাইডিং আউটপুট প্রশ্ন ★

সংজ্ঞা ওভাররাইডিং *(Overriding)* = ডিরাইভড ক্লাস উত্তরাধিকারসূত্রে পাওয়া মেথডকে **একই সিগনেচারে** নতুন করে লেখে। *C++*-এর ভার্চুয়াল ফাংশন *(Virtual function)* পয়েন্টারের নয়, **অবজেক্টের** টাইপ দেখে ঠিক হয়, তাই `Base* p = new Derived; p->f();` চালায় `Derived::f`। `virtual` না থাকলে পয়েন্টারের টাইপই ঠিক করে (স্ট্যাটিক বাইন্ডিং)। কম্পাইলার এটি করে প্রতি ক্লাসে একটি **vtable** ও প্রতি অবজেক্টে একটি লুকানো **vptr** দিয়ে।

```
class Base { public:
  void show() { cout << "Base "; }
  virtual void print() { cout << "BasePrint"; }
};
class Derived : public Base { public:
  void show() { cout << "Derived "; }
  void print() override
              { cout << "DerivedPrint"; }
};
Base* p = new Derived;
p->show();   // Base          (static)
p->print();  // DerivedPrint (dynamic)
// output: Base DerivedPrint
```



p->print() আগে vptr পড়ে, তারপর vtable হয়ে লাফ দেয়

ক্লাসপ্রতি একটি *vtable*, অবজেক্টপ্রতি একটি *vptr*

বিষয়	ওভারলোডিং	ওভাররাইডিং
কোথায়	একই ক্লাসে	বেস ও ডিরাইভড ক্লাসে
সিগনেচার	**ভিন্ন** হতে হবে	**একই** হতে হবে
রিটার্ন টাইপ	যেকোনো	একই বা কোভেরিয়েন্ট (সাবক্লাস টাইপ)
বাইন্ডিং	কম্পাইল টাইম	রান টাইম
অ্যাক্সেস *(Java)*	যেকোনো	একই বা বেশি, কখনো কম নয়
static / private / final	ওভারলোড করা যায়	**ওভাররাইড হয় না**; *static* হাইড হয়

ভাষা	ডিফল্টভাবে ভার্চুয়াল?	কিওয়ার্ড	ওভাররাইড বন্ধ
C++	না	`virtual`, `override`	`final` *(C++11)*
Java	**হ্যাঁ** *(non-static, non-private)*	`@Override` (ঐচ্ছিক)	`final`
C#	না	`virtual` / `override`	`sealed`

মনে রাখার কৌশল "পয়েন্টার মেনু বাছে; `virtual` থাকলে অবজেক্ট পদ বাছে।"

পরীক্ষার ফাঁদ কনস্ট্রাক্টর ভার্চুয়াল হতে পারে না; কনস্ট্রাক্টরের ভেতরে ভার্চুয়াল কল বেস ভার্সন চালায়। *Java*-তে ওভাররাইড করে **অ্যাক্সেস কমানো** বা বড় *checked* এক্সসেপশন ছোড়া যায় না।

সমাধানকৃত উদাহরণ ✍

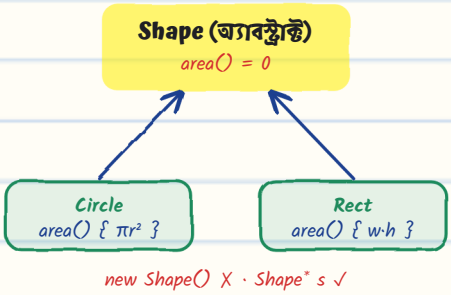
sizeof

৬৪-বিট: `class A { int x; };` → 4। একটি ভার্চুয়াল ফাংশন যোগ করলে → 16 (8-বাইট *vptr* + 4 + প্যাডিং)। আরও ভার্চুয়াল ফাংশনেও *vptr* একটিই।

৩ অ্যাবস্ট্রাক্ট ক্লাস ও ভার্চুয়াল ডেস্ট্রাক্টর

হাই-ইল্ড ★

সংজ্ঞা পিওর ভার্চুয়াল ফাংশন *(Pure virtual function)* ঘোষণা করা হয় `= 0` দিয়ে — `virtual double area() = 0;` — অর্থাৎ "প্রতিটি সাবক্লাসকে এটি দিতেই হবে"। **অন্তত একটি** পিওর ভার্চুয়াল ফাংশনযুক্ত ক্লাস হলো **অ্যাবস্ট্রাক্ট ক্লাস *(Abstract class)***: এর অবজেক্ট তৈরি করা যায় না, তবে পয়েন্টার ও রেফারেন্স রাখা যায়। যে ডিরাইভড ক্লাস সব পিওর ভার্চুয়াল ফাংশন ওভাররাইড করে না, সেটিও অ্যাবস্ট্রাক্ট। *Java* ও *C#* ব্যবহার করে `abstract` কিওয়ার্ড।



ড্যাশ দাগ = ইনস্ট্যান্স হয় না

```
class Shape { public:
  virtual double area() = 0;  // pure
  virtual ~Shape() {}         // virtual dtor
};
class Circle : public Shape { double r;
public:
  Circle(double r) : r(r) {}
  double area() override { return 3.14159*r*r; }
};
Shape* s[2] = { new Circle(1), new Rect(2, 3) };
double total = 0;
for (auto x : s) total += x->area();
// Shape t;  → error: abstract class
```

অ্যাবস্ট্রাক্ট ক্লাসের কি ...	C++	Java
কনস্ট্রাক্টর, ডেটা, সাধারণ মেথড থাকতে পারে?	হ্যাঁ (কনস্ট্রাক্টর চালায় ডিরাইভড ক্লাস)	হ্যাঁ
শূন্য অ্যাবস্ট্রাক্ট মেথড থাকতে পারে?	না — ≥ ১টি পিওর ভার্চুয়াল হলেই অ্যাবস্ট্রাক্ট	হ্যাঁ, `abstract` লিখলে
অ্যাবস্ট্রাক্ট মেথডের বডি দেওয়া যায়?	হ্যাঁ, বাইরে: `Shape::area()`	না

Java-তে একই `abstract class Shape { abstract double area(); }` · `class Circle extends Shape {…}` · `Shape s = new Circle(1);` বৈধ — রেফারেন্সের টাইপ অ্যাবস্ট্রাক্ট হতে পারে।

ভার্চুয়াল ডেস্ট্রাক্টর `Base* p = new Derived; delete p;` — `~Base()` ভার্চুয়াল **না** হলে কেবল বেস ডেস্ট্রাক্টর চলে (আনুষ্ঠানিকভাবে *undefined behaviour*), ডিরাইভড অংশ লিক হয়। নিয়ম: যে ক্লাসে কোনো ভার্চুয়াল ফাংশন আছে, তার ডেস্ট্রাক্টর ভার্চুয়াল হবে। তখন ক্রম: `~Derived()` তারপর `~Base()`।

অবজেক্ট স্লাইসিং (Object slicing) `Derived d; Base b = d;` কেবল ***Base* অংশ** কপি করে; ডিরাইভড ফিল্ড "কেটে বাদ" যায় এবং `b.print()` ডাকে `Base::print`। পলিমরফিজম কাজ করে কেবল **পয়েন্টার বা রেফারেন্সে**, ভ্যালুতে কখনো নয়। *Java*-তে স্লাইসিং নেই: ভেরিয়েবল রেফারেন্স রাখে।

মনে রাখার কৌশল "**`= 0` মানে শূন্য অবজেক্ট।**" ক্লাসে কোথাও `= 0` বসালে আর অবজেক্ট বানানো যায় না।

পরীক্ষার ফাঁদ "অ্যাবস্ট্রাক্ট ক্লাসে কেবল অ্যাবস্ট্রাক্ট মেথড থাকে" — **ভুল**। *C++*-এ `abstract` কিওয়ার্ড নেই। অ্যাবস্ট্রাক্ট ক্লাস বনাম ইন্টারফেস: প্রথম লেসন।

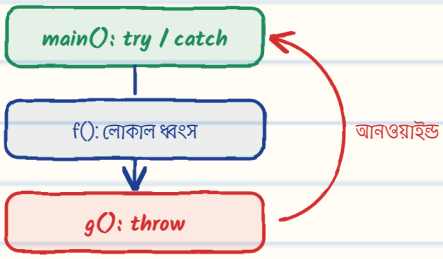
সমাধানকৃত উদাহরণ ✍ মোট ক্ষেত্রফল

Circle(1) + Rect(2,3) = 3.14159 + 6 = 9.14 প্রতিটি `area()` রান টাইমে বাছাই।

৪ এক্সসেপশন হ্যান্ডলিং: C++ ও Java আউটপুট প্রশ্ন ★

সংজ্ঞা এক্সসেপশন (Exception) হলো রান টাইমের অস্বাভাবিক ঘটনা (শূন্য দিয়ে ভাগ, ভুল ইনডেক্স, ফাইল নেই) যা স্বাভাবিক প্রবাহ ভাঙে। try ব্লক পাহারা দেয়, throw এক্সসেপশন অবজেক্ট ছোড়ে, catch মিলে যাওয়া টাইপ সামলায়। throw হলে শুরু হয় স্ট্যাক আনওয়াইন্ডিং (Stack unwinding): হ্যান্ডলার না মেলা পর্যন্ত ফ্রেম সরানো হয় ও লোকাল অবজেক্ট ধ্বংস হয়; কোনো হ্যান্ডলার না পেলে → std::terminate() (C++) বা স্ট্যাক ট্রেসসহ থ্রেড বন্ধ (Java)।

```
double divide(int a, int b) {
  if (b == 0) throw runtime_error("b = 0");
  return (double)a / b;
}
try {
  cout << divide(10, 0);   // skipped
} catch (const runtime_error& e) {
  cout << e.what();        // b = 0
} catch (...) {            // any type
  cout << "unknown";
}
```



স্ট্যাক আনওয়াইন্ডিং

কিওয়ার্ড	C++	Java
throw	যেকোনো টাইপ; throw; = পুনরায় ছোড়া	throw new X() — একটি Throwable
throws	নেই; noexcept = ছুড়বে না	হেডারে checked এক্সসেপশনের তালিকা
finally	**নেই** — ডেস্ট্রাক্টর পরিষ্কার করে (RAII)	এক্সসেপশন **হোক বা না হোক** চলে
সব ধরা	catch (...), সবশেষে	catch (Exception e); মাল্টি-ক্যাচ A | B

Java ক্লাস	ধরন	উদাহরণ
Throwable → **Error**	unchecked	OutOfMemoryError, StackOverflowError (ধরবে না)
Exception → **RuntimeException**	unchecked	Arithmetic, NullPointer, ArrayIndexOutOfBounds, ClassCast
অন্যান্য **Exception**	checked	IOException, FileNotFound, SQLException, InterruptedException

Checked = কম্পাইলার বাধ্য করে catch করতে বা throws দিয়ে ঘোষণা করতে। C++: std::exception → logic_error (out_of_range, invalid_argument), runtime_error (overflow_error, underflow_error), bad_alloc, bad_cast; সবার আছে what()।

মনে রাখার কৌশল "**সন্তানকে ধরো বাবার আগে।**" প্রথম মিলে যাওয়া হ্যান্ডলারই জেতে, তাই ডিরাইভড টাইপ আগে। "**Checked = Compiler-checked।**"

পরীক্ষার ফাঁদ সাবক্লাস catch-এর ওপরে catch (Exception e) = Java-তে **কম্পাইল এরর**। finally বাদ যায় কেবল System.exit() বা ক্র্যাশে; finally-র return try-এর return বাতিল করে।

সমাধানকৃত উদাহরণ ✍ ট্রেস

```
int[] a = new int[3];
try { a[3]=10;
print("X"); } catch
{print("Y");} finally
{print("Z");}
print("W"); → YZW
```

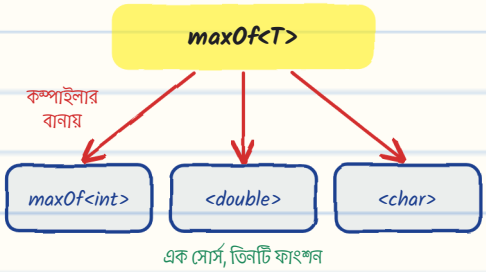
৫ টেমপ্লেট ফাংশন ও ক্লাস; Java জেনেরিক্স প্রোগ্রামিং প্রশ্ন ★

সংজ্ঞা টেমপ্লেট (*Template*) হলো টাইপ প্যারামিটার T দিয়ে একবার লেখা ব্লুপ্রিন্ট। কোড যখন কোনো নির্দিষ্ট টাইপ দিয়ে এটি ব্যবহার করে, কম্পাইলার তখন একে ইনস্ট্যান্সিয়েট (*Instantiate*) করে — সেই টাইপের জন্য আলাদা ফাংশন বা ক্লাস তৈরি করে — **কম্পাইল টাইমে**। এটি **প্যারামেট্রিক (জেনেরিক) পলিমরফিজম**; পুরো *STL* (`vector<T>`, `sort`, `map<K,V>`) এর ওপর তৈরি।

```
template <typename T>
T maxOf(T a, T b) { return a > b ? a : b; }

cout << maxOf(3, 7)        // T = int
     << " " << maxOf(2.5, 1.5)
     << " " << maxOf('a', 'z');
// output: 7 2.5 z
// maxOf(3, 2.5)  → error: T int or double?
// maxOf<double>(3, 2.5) → 3
```

```
template <class T, int N = 100>
class Stack {
  T data[N];  int top = -1;
public:
  void push(const T& x) {
    if (top == N - 1) throw overflow_error("full");
    data[++top] = x;
  }
  T pop() {
    if (top < 0) throw underflow_error("empty");
    return data[top--];
  }
};
Stack<int> si;  Stack<string, 10> ss;
```



কম্পাইল টাইমে ইনস্ট্যান্সিয়েশন

স্পেশালাইজেশন (Specialization) **ফুল স্পেশালাইজেশন** একটি নির্দিষ্ট টাইপের জন্য টেমপ্লেট বদলে দেয়: `template<> const char* maxOf(const char* a, const char* b)`, ভেতরে `strcmp` — না দিলে দুটি C স্ট্রিং তুলনা হয় **ঠিকানা** দিয়ে। ক্লাস টেমপ্লেট **আংশিক** স্পেশালাইজ করা যায় (`Stack<T*>`); ফাংশন টেমপ্লেট নয়। টেমপ্লেট কোড থাকে **হেডারে**: ইনস্ট্যান্সিয়েট করতে কম্পাইলারের পুরো বডি লাগে।

বিষয়	C++ টেমপ্লেট	Java জেনেরিক্স
নির্ধারণ	কম্পাইল টাইমে, টাইপপ্রতি	কম্পাইল টাইমে, তারপর **টাইপ ইরেজার**
কোডের কপি	টাইপপ্রতি একটি (কোড ব্লোট)	একটি ক্লাস (ভেতরে `Object`)
প্রিমিটিভ আর্গুমেন্ট	হ্যাঁ: `Stack<int>`	না: `List<Integer>`, কখনো `<int>` নয়
বাড়তি সুবিধা	নন-টাইপ প্যারাম `<int N>`, স্পেশালাইজেশন	বাউন্ড `<T extends Comparable<T>>`, ওয়াইল্ডকার্ড `?`

মনে রাখার কৌশল "**টেমপ্লেট হলো কুকি কাটার**": এক কাটার, প্রতিটি ময়দার (টাইপের) জন্য নতুন কুকি। *Java* জেনেরিক্স: একটিই কুকি, গায়ে লেবেল।

পরীক্ষার ফাঁদ *Java* জেনেরিক্স টেমপ্লেট **নয়**: ইরেজারের পর `List<String>` ও `List<Integer>` **একই ক্লাস**, তাই `new T()` অবৈধ।

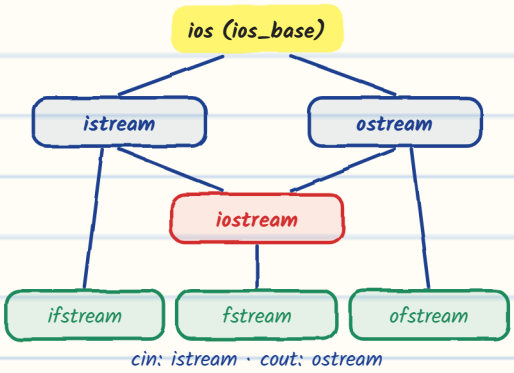
সমাধানকৃত উদাহরণ ✍

Java

`static <T> void show(T v)`: `show(100)` অটোবক্স হয়ে *Integer*। `Box<String> b = new Box<>();` — ডায়মন্ড *String* অনুমান করে।

৬ অবজেক্ট ওরিয়েন্টেড I/O: স্ট্রিম MCQ তথ্য

সংজ্ঞা OO I/O-তে ডেটা চলে স্ট্রিম অবজেক্টের (Stream object) মাধ্যমে: স্ট্রিম হলো বাইটের ধারা, যা উৎস (কিবোর্ড, ফাইল) থেকে আসে বা গন্তব্যে (স্ক্রিন, ফাইল) যায়। C++ ক্লাস হায়ারার্কি দেয় <iostream>, <fstream>, <sstream>-এ; << (ইনসার্শন) ও >> (এক্সট্র্যাকশন) ওভারলোডেড অপারেটর, তাই ইউজার ক্লাসও যোগ দিতে পারে।



iostream: মাল্টিপল ইনহেরিট্যান্স

অবজেক্ট	ক্লাস	ডিভাইস	বাফার
cin	istream	কিবোর্ড	হ্যাঁ
cout	ostream	স্ক্রিন	হ্যাঁ
cerr	ostream	এরর	না
clog	ostream	এরর লগ	হ্যাঁ

মোড	অর্থ
ios::in · out	পড়া · লেখা
ios::app	প্রতিটি লেখা শেষে যোগ
ios::ate	খুলে একবার শেষে যাও
ios::trunc	আগে ফাইল খালি করো
ios::binary	টেক্সট রূপান্তর নয়

```
class Point { int x, y;
 public:
  Point(int a=0, int b=0) : x(a), y(b) {}
  friend ostream& operator<<(ostream& os,
                             const Point& p) {
    return os << "(" << p.x << ","
              << p.y << ")";
  }
};
ofstream out("pts.txt", ios::app);
out << Point(2, 3) << endl;  // (2,3)
out.close();
ifstream in("pts.txt");  string line;
while (getline(in, line)) cout << line;
```

ম্যানিপুলেটর	কাজ
endl	নতুন লাইন + ফ্লাশ
setw(n)	প্রস্থ, কেবল পরের আইটেমে
setprecision(n)	অঙ্ক (fixed হলে দশমিকের পরে)
setfill(c)	প্যাডিং অক্ষর
hex oct dec	সংখ্যার ভিত্তি (স্থায়ী)

ofstream ডিফল্টে খোলে out | trunc মোডে; ফাইল যাচাই is_open(), eof(), fail() দিয়ে।

Java I/O **বাইট** স্ট্রিম: InputStream / OutputStream (FileInputStream)। **ক্যারেক্টার** স্ট্রিম: Reader / Writer (FileReader, FileWriter, readLine()-সহ BufferedReader)। System.in একটি InputStream, System.out একটি PrintStream; Scanner টোকেন পার্স করে (nextInt(), nextLine())। ফাইল বন্ধ করো ***try-with-resources*** দিয়ে; পুরো অবজেক্ট সেভ হয় ObjectOutputStream-এ (ক্লাসকে Serializable হতে হবে)।

মনে রাখার কৌশল "**তীর দেখায় ডেটা কোথায় যায়**": cout << x x-কে বাইরে ঠেলে; cin >> x x-এ টেনে আনে।

পরীক্ষার ফাঁদ operator<< হবে **নন-মেম্বার** (friend), চেইনিংয়ের জন্য রিটার্ন ostream&। cin >> s স্পেসে থেমে যায়।

সমাধানকৃত উদাহরণ

cout << setw(5) << 42 << 7; → "...427": setw কেবল 42-কে প্যাড করে।

৭ থ্রেড: তৈরি ও জীবনচক্র হাই-ইল্ড ★

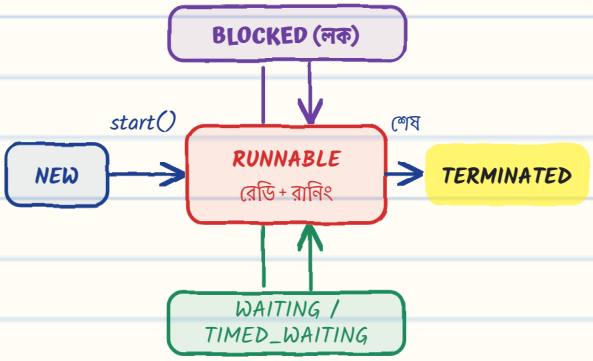
সংজ্ঞা থ্রেড (Thread) হলো CPU শিডিউলিংয়ের ক্ষুদ্রতম একক — একটি "লাইটওয়েট প্রসেস"। মাল্টি-থ্রেডিং (Multi-threading) এক প্রসেসের ভেতরে দুই বা ততোধিক থ্রেড চালায়; তারা কোড, ডেটা ও হিপ ভাগ করে, কিন্তু প্রত্যেকের নিজস্ব **স্ট্যাক, রেজিস্টার ও প্রোগ্রাম কাউন্টার**। এক কোরে থ্রেড চলে **সমকালীনভাবে (concurrent)**, পালা করে; বহু কোরে **সমান্তরালে (parallel)**। ব্যবহার: সাড়া-দেওয়া GUI, সার্ভার, ব্যাকগ্রাউন্ড ডাউনলোড।

বিষয়	প্রসেস	থ্রেড
মেমরি	নিজস্ব অ্যাড্রেস স্পেস	প্রসেসের মেমরি ভাগ করে
তৈরি / সুইচ	ভারী, ধীর	হালকা, দ্রুত
যোগাযোগ	IPC: পাইপ, সকেট	শেয়ার্ড ভেরিয়েবল (লক লাগে)
একটি ক্র্যাশ করলে	অন্যরা অক্ষত	পুরো প্রসেস বন্ধ হতে পারে

```
// Way 1: extend Thread
class MyThread extends Thread {
  public void run() { System.out.println("T1"); }
}
// Way 2: implement Runnable (preferred)
class Task implements Runnable {
  public void run() { System.out.println("T2"); }
}
new MyThread().start();
new Thread(new Task()).start();
new Thread(() -> System.out.println("T3"))
    .start();              // lambda (Java 8)
```

start() বনাম run() start() JVM-এর কাছে **নতুন** থ্রেড চায়, সেই থ্রেড পরে run() ডাকে। সরাসরি run() ডাকা **বর্তমান** থ্রেডে সাধারণ মেথড কল। দুবার start() → IllegalThreadStateException। **Runnable ভালো**: ক্লাসটি তখনও অন্য ক্লাস extend করতে পারে (একক ইনহেরিট্যান্স), আর একটি টাস্ক বহু থ্রেড ভাগ করতে পারে। থ্রেডগুলোর প্রিন্টের ক্রম **নিশ্চিত নয়**।

মেথড	কাজ
sleep(ms)	থামে; লক **রাখে**
join()	ওটা শেষ হওয়া পর্যন্ত অপেক্ষা
yield()	ইঙ্গিত: অন্যকে চলতে দাও
wait()	লক **ছেড়ে দেয়**
setPriority	1 … 5 … 10
setDaemon	ব্যাকগ্রাউন্ড থ্রেড



Java Thread.State: ছয়টি অবস্থা

মনে রাখার কৌশল "New Runners Block, Wait, Time-wait, Terminate" — enum-এর ক্রমে Java-র ছয় অবস্থা।

পরীক্ষার ফাঁদ Java-তে **আলাদা RUNNING অবস্থা নেই**: RUNNABLE = রেডি + রানিং। লকের অপেক্ষা = BLOCKED; wait()/join() = WAITING; sleep(ms) = TIMED_WAITING।

সমাধানকৃত উদাহরণ

t.run(); print("M"); → run-এর আউটপুট সবসময় আগে (একই থ্রেড)। t.start(); print("M"); → ক্রম অনিশ্চিত।

৮ সিনক্রোনাইজেশন, রেস কন্ডিশন ও ডেডলক

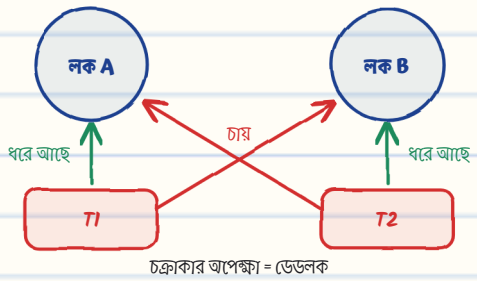
লিখিত প্রশ্নের প্রিয় ★

সংজ্ঞা রেস কন্ডিশন *(Race condition)* ঘটে যখন দুটি থ্রেড শেয়ার্ড ডেটা ব্যবহার করে, অন্তত একটি লেখে, আর ফল সময়ের ওপর নির্ভর করে। শেয়ার্ড ডেটা ছোঁয়া কোড হলো ক্রিটিক্যাল সেকশন *(Critical section)*; **সিনক্রোনাইজেশন** নিশ্চিত করে **মিউচুয়াল এক্সক্লুশন** — একসঙ্গে কেবল একটি থ্রেড ভেতরে। `count++` দেখতে এক ধাপ, আসলে তিন ধাপ (পড়া, যোগ, লেখা), তাই দুটি থ্রেড একটি আপডেট হারাতে পারে।

```java
// Java: monitor lock on 'this'
class Counter {
  private int count = 0;
  public synchronized void increment() {
    count++;
  }
  public int get() { return count; }
}
// or: synchronized (lock) { count++; }
// or: AtomicInteger c; c.incrementAndGet();
```

```cpp
// C++11: <thread>, <mutex>
int count = 0;  mutex m;
void work() {
  for (int i = 0; i < 1000; i++) {
    lock_guard<mutex> g(m);  // RAII lock
    count++;
  }
}
thread t1(work), t2(work);
t1.join();  t2.join();   // wait for both
cout << count;           // always 2000
```

হাতিয়ার	যা দেয়
মিউটেক্স *(Mutex)*	একসময়ে এক মালিক; যে লক করে, সে-ই খোলে
সেমাফোর *(Semaphore)*	কাউন্টার: সর্বোচ্চ *n* থ্রেড; বাইনারি সেমাফোর ≈ মিউটেক্স (মালিকানা ছাড়া)
মনিটর *(Monitor)*	এক অবজেক্টে লক + অপেক্ষা: `synchronized` + `wait/notify`
`volatile` *(Java)*	সর্বশেষ মান দৃশ্যমান রাখে; **অ্যাটমিসিটি নয়** — `count++` তবুও রেস করে
`join` / `detach` *(C++)*	অপেক্ষা করো / একা চলতে দাও; কোনোটি না করলে → `std::terminate`



প্রত্যেকে একটি ধরে, অন্যটির অপেক্ষায়

ডেডলক: Coffman-এর ৪ শর্ত ① **মিউচুয়াল এক্সক্লুশন** — রিসোর্স একটি থ্রেডের হাতে। ② **হোল্ড অ্যান্ড ওয়েট** — একটি ধরে আরেকটির অপেক্ষা। ③ **নো প্রিএম্পশন** — লক কেড়ে নেওয়া যায় না। ④ **সার্কুলার ওয়েট** — *T1 → T2 → … → T1*। চারটিই লাগে; একটি ভাঙলেই ডেডলক *(Deadlock)* অসম্ভব — সাধারণত **নির্দিষ্ট লক-ক্রম** (সবসময় *A* আগে, পরে *B*)। **স্টারভেশন** = কখনো পালা পায় না; **লাইভলক** = ব্যস্ত কিন্তু অগ্রগতি নেই।

মনে রাখার কৌশল *Coffman = "**My Hat, No Coat**": Mutual exclusion, Hold & wait, No preemption, Circular wait।*

পরীক্ষার ফাঁদ `wait()`/`notify()` হলো ***Object***-এর মেথড, লক হাতে রেখেই ডাকতে হয় (নইলে *IllegalMonitorStateException*)।

সমাধানকৃত উদাহরণ ✍️

হারানো আপডেট

count = 5। T1 পড়ে 5, T2 পড়ে 5, T1 লেখে 6, T2 লেখে 6 → ফল 6, 7 নয়। লক ছাড়া দুই × ১০০০ বৃদ্ধিতে ফল ≤ 2000।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা

ওভারলোডিং → কম্পাইল টাইম, আর্লি বাইন্ডিং
শুধু রিটার্ন টাইপে ওভারলোড হয় না
কেবল মেম্বার: = () [] ->
vtable ক্লাসপ্রতি, *vptr* অবজেক্টপ্রতি
`virtual f() = 0` → অ্যাবস্ট্রাক্ট ক্লাস
C++-এ finally নেই (RAII); Java-র finally সবসময় (System.exit ছাড়া)
Unchecked: RuntimeException + Error
Java জেনেরিক্স: টাইপ ইরেজার, প্রিমিটিভ নয়
থ্রেড হিপ ভাগ করে; নিজস্ব স্ট্যাক + PC

ওভাররাইডিং + *virtual* → রান টাইম, লেট বাইন্ডিং
ওভারলোড নয়: `:: . .* ?: sizeof`
Java: ইউজার অপারেটর ওভারলোডিং নেই
কনস্ট্রাক্টর ভার্চুয়াল নয়; ডেস্ট্রাক্টর হওয়া উচিত
পলিমরফিজম পয়েন্টার/রেফারেন্সে; ভ্যালুতে = স্লাইসিং
Checked: IOException, SQLException

টেমপ্লেট: কম্পাইল টাইমে টাইপপ্রতি তৈরি
`endl` = নতুন লাইন + ফ্লাশ; `cerr` বাফারহীন
Java: ৬টি থ্রেড অবস্থা; প্রায়োরিটি *1–10*, ডিফল্ট *5*

→ MCQ-র সেরা ১২ এক-লাইন

1. পলিমরফিজম = এক ইন্টারফেস, অনেক রূপ
2. ফাংশন ওভারলোডিং = স্ট্যাটিক পলিমরফিজম
3. *Java*-র মেথড ডিফল্টভাবে ভার্চুয়াল
4. *C#*-এ লাগে `virtual` + `override`; `sealed` থামায়
5. ডিরাইভড এক্সসেপশন বেসের আগে *catch*
6. `catch (...)` সব ধরে, থাকে শেষে
7. `throw` ছোড়ে; `throws` ঘোষণা করে
8. `noexcept` ভাঙলে → `std::terminate`
9. *STL* টেমপ্লেটের ওপর তৈরি
10. `<<` ওভারলোড = *friend*, রিটার্ন `ostream&`
11. `start()` নতুন থ্রেড; `run()` একই থ্রেড
12. ডেডলকে *Coffman*-এর চারটি শর্তই লাগে

প্রধান ফাঁদ রিটার্ন টাইপ ≠ ওভারলোড · কনস্ট্রাক্টরে ভার্চুয়াল কল বেস ভার্সন চালায় · অ্যাবস্ট্রাক্ট ক্লাসে কনস্ট্রাক্টর থাকতে পারে · *Java*-তে *RUNNING* অবস্থা নেই · `volatile` ≠ অ্যাটমিক · `sleep()` লক রাখে, `wait()` ছাড়ে · সাবক্লাসের আগে *catch(Exception)* = কম্পাইল এরর।

সব কৌশল একসঙ্গে *Scope, Dot, Dot-star, Question, Size* · পয়েন্টার মেনু বাছে, অবজেক্ট পদ বাছে · *= 0* মানে শূন্য অবজেক্ট · সন্তানকে ধরো বাবার আগে · কুকি কাটার · তীর দেখায় ডেটা কোথায় যায় · *New Runners Block…* · *My Hat, No Coat*।



কখন ঠিক হয়?

সমাধানকৃত উদাহরণ ✍ একটি কিওয়ার্ডে আউটপুট বদল *Base/Derived* প্রোগ্রামে `print()` থেকে `virtual` সরাও: `p->print()` এখন পয়েন্টারের টাইপে বাঁধা, তাই আউটপুট *"Base DerivedPrint"*-এর বদলে হয় *"Base BasePrint"*।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে চেষ্টা, পরে মেলাও

১. ফাংশন ওভারলোডিং নির্ধারিত হয়:
ক) কম্পাইল টাইমে খ) রান টাইমে
গ) লিংক টাইমে ঘ) লোড টাইমে

২. C++-এ কোন অপারেটর ওভারলোড করা যায় না?
ক) + খ) []
গ) == ঘ) ::

৩. অন্তত একটি পিওর ভার্চুয়াল ফাংশনযুক্ত C++ ক্লাস হলো:
ক) ফ্রেন্ড ক্লাস খ) টেমপ্লেট ক্লাস
গ) অ্যাবস্ট্রাক্ট ক্লাস ঘ) স্ট্যাটিক ক্লাস

৪. Java-তে এক্সসেপশন হোক বা না হোক, কোন ব্লক চলে?
ক) catch খ) finally
গ) throw ঘ) throws

৫. Java-তে নতুন থ্রেড শুরু করতে ডাকতে হয়:
ক) run() খ) init()
গ) start() ঘ) execute()

৬. Java-তে কোনটি unchecked এক্সসেপশন?
ক) IOException খ) SQLException
গ) EOFException ঘ) ArithmeticException

৭. বেস ক্লাসের ডেস্ট্রাক্টর ভার্চুয়াল করা হয় কেন?
ক) Base* দিয়ে delete করলে ~Derived চলবে বলে খ) ক্লাসকে অ্যাবস্ট্রাক্ট করতে
গ) কপি আটকাতে ঘ) delete দ্রুত করতে

৮. C++ টেমপ্লেট ইনস্ট্যান্সিয়েট হয়:
ক) রান টাইমে খ) কম্পাইল টাইমে
গ) লোড টাইমে ঘ) কেবল লিংক টাইমে

৯. Java-র Thread.State-এ কয়টি অবস্থা?
ক) 6 খ) 4
গ) 5 ঘ) 7

১০. কোনটি ডেডলকের আবশ্যিক শর্ত **নয়**?
ক) মিউচুয়াল এক্সক্লুশন খ) হোল্ড অ্যান্ড ওয়েট
গ) সার্কুলার ওয়েট ঘ) প্রিএম্পশন

উত্তরমালা → ১-ক · ২-ঘ · ৩-গ · ৪-খ · ৫-গ · ৬-ঘ · ৭-ক · ৮-খ · ৯-ক · ১০-ঘ

লিখিত প্রশ্ন কম্পাইল-টাইম ও রান-টাইম পলিমরফিজমের পার্থক্য লেখো। উদাহরণ ও vtable কৌশলসহ ব্যাখ্যা করো, C++-এ ভার্চুয়াল ফাংশন কীভাবে রান-টাইম পলিমরফিজম দেয়। পলিমরফিক বেস ক্লাসের ডেস্ট্রাক্টর কেন ভার্চুয়াল হওয়া উচিত?

১০ নম্বর

১ **পার্থক্য (৩)** — কম্পাইল-টাইম: ওভারলোডিং / টেমপ্লেট, আর্গুমেন্ট তালিকা দেখে বাছাই, আর্লি বাইন্ডিং, দ্রুত; রান-টাইম: বেস পয়েন্টার বা রেফারেন্সের মাধ্যমে ভার্চুয়াল ফাংশনের ওভাররাইডিং, অবজেক্টের টাইপ দেখে বাছাই, লেট বাইন্ডিং, ইনহেরিট্যান্স লাগে। ৩

২ **উদাহরণ (৩)** — `virtual void print()`-সহ Base, যা Derived ওভাররাইড করে; `Base* p = new Derived; p->print();` ডিরাইভড ভার্সন প্রিন্ট করে; `virtual` ছাড়া বেস ভার্সন। ৩

৩ **কৌশল (২)** — ভার্চুয়াল ফাংশনযুক্ত প্রতিটি ক্লাসের ফাংশন-ঠিকানার একটি vtable; প্রতিটি অবজেক্টে নিজ ক্লাসের vtable-এর দিকে লুকানো vptr; কল vptr পড়ে, টেবিলে ইনডেক্স করে লাফ দেয়। ২

৪ **ভার্চুয়াল ডেস্ট্রাক্টর (২)** — নন-ভার্চুয়াল ~Base-এ `delete p` কেবল বেস ডেস্ট্রাক্টর চালায় (undefined behaviour, লিক); ভার্চুয়াল হলে ~Derived তারপর ~Base। ২

→ পরীক্ষার আগে ✓

- ✓ ওভারলোড-অযোগ্য অপারেটরগুলো বলতে পারি
- ✓ ভার্চুয়াল / নন-ভার্চুয়াল কল ট্রেস করতে পারি
- ✓ C++ ও Java-তে অ্যাবস্ট্রাক্ট ক্লাস লিখতে পারি
- ✓ catch সাজাতে ও finally ট্রেস করতে পারি
- ✓ ফাংশন ও ক্লাস টেমপ্লেট লিখতে পারি
- ✓ ক্লাসের জন্য << ওভারলোড করতে পারি
- ✓ Java-র ছয় থ্রেড অবস্থা আঁকতে পারি
- ✓ ডেডলকের চার শর্ত বলতে পারি